RAPPORT D'ARCHITECTURE TECHNIQUE

Sujet :	Arbitrage HFT sous cTrader Open API en Rust
Environnement :	Linux low-jitter (10µs), Centre de données Métropolitain (Londres)
Cible Temporelle :	Fenêtre d'opportunité de 150 ms (Détention de position)
Contrainte LP :	Fenêtre de Last Look asymétrique fixe de 40 ms

> **Synthèse exécutive :** Ce document définit les spécifications et choix d'ingénierie logicielle requis pour exploiter une stratégie d'arbitrage de latence de 150 ms. Face à un mécanisme de Last Look de 40 ms appliqué par les Fournisseurs de Liquidité (LP), l'application cliente doit éliminer toute attente synchrone via un modèle découplé asynchrone développé en Rust, tout en forçant des contraintes d'exécution strictes au niveau du protocole binaire.

1. Analyse Chronométrique et Budget de Latence

Pour une fenêtre d'opportunité globale de 150 ms, la latence de transit aller simple (One-Way) et le temps de traitement de l'ordre par les contreparties consomment une part incompressible de votre marge temporelle. L'architecture est modélisée avec un serveur Linux optimisé en dehors du data center principal de cTrader (ex. Equinix LD4 Slough), induisant des coûts de routage réseau métropolitain.

Segment d'Infrastructure / Couche de Traitement	Latence Minimale	Latence Maximale	Type d'Impact Applicatif
Couche 1 : Sérialisation Rust Protobuf + Socket Kernel	0,02 ms	0,05 ms	Déterministe (Jitter OS < 10µs)
Couche 2 : Réseau fibre externe + Routage Edge Cloud	0,40 ms	1,50 ms	Variable (Gigue de transport réseau)
Couche 3 : Moteur de Risque du Courtier (In-Memory)	0,10 ms	0,30 ms	Dépend de la charge globale du broker
Couche 4 : Conversion FIX + Routage LP (Last Look)	40,10 ms	40,50 ms	Fixe (Délai de décision asymétrique du LP)
Total Cumulé (Temps d'Arrivée / Remplissage)	**40,62 ms**	**42,35 ms**	**Budget consommé : ~28% de la fenêtre**

Analyse du Budget Résiduel : Une fois l'ordre exécuté (Statut `FILLED`) à T + 42 ms, le système ne dispose plus que de **~107 ms** de validité structurelle avant que l'anomalie de prix ne se referme. À cela doit être soustrait le temps de transit de l'ordre de sortie (fermeture de position), qui subira les mêmes contraintes réseaux et un potentiel second Last Look de 40 ms. Le temps d'exposition réel au marché est donc limité à un maximum théorique de **65 ms**.

2. Architecture Logique en Rust : Découplage Synchrone / Asynchrone

Pour préserver l'intégrité du moteur de détection d'anomalies, le thread principal de calcul ne doit jamais être bloqué par l'attente du flux réseau TCP (I/O Bound). L'attente des 40 ms imposée par le Last Look du LP est gérée de manière totalement transparente par un modèle asynchrone structuré autour de tâches distribuées (Tokio Green Threads) interconnectées par des canaux à passage de messages (MPSC channels).

Le mécanisme de Corrélation Asynchrone (client_msg_id)

Puisque le socket réseau envoie des rafales de requêtes en continu, le serveur cTrader désynchronise les réponses en fonction du temps de réponse de chaque LP. Pour corréler la réponse binaire reçue au bout de 40 ms avec l'ordre d'arbitrage initial, l'application Rust exploite le champ natif du protocole ouvert cTrader : `client_msg_id`.

- À chaque émission de message `ProtoOANewOrderReq`, un identifiant de corrélation alphanumérique unique et compact est généré (ex: horodatage haute résolution combiné à un index atomique).
- Cet identifiant est simultanément inséré dans une table de hachage thread-safe concurrente (`DashMap` ou une structure de dictionnaire protégée par un `Mutex` ou `RwLock`) contenant les métadonnées de l'état du trade.
- La tâche réseau de fond (Receiver Event Loop) écoute les trames entrantes. Dès réception d'un événement d'exécution, elle extrait le `client_msg_id` sans désérialiser l'intégralité du paquet, effectue une recherche en temps constant $O(1)$ dans la table, et réveille la logique métier dédiée à ce trade précis.

3. Exploitation du "Pre-Fill Approval" et Reroutage ECN

La Couche 4 (le *Liquidity Bridge* du courtier) implémente des algorithmes de routage intelligents. Si un ordre est transmis avec des contraintes d'exécution strictes, le Bridge est capable d'intercepter les rejets rapides de certains LPs (qui peuvent survenir en quelques millisecondes si le LP refuse le flux toxique d'arbitrage) pour tenter d'obtenir une exécution immédiate auprès d'une contrepartie alternative.

Pour activer ce mécanisme de protection et empêcher qu'un ordre ne reste bloqué en attente chez un LP défaillant, deux paramètres fondamentaux du protocole doivent être configurés conjointement :

- **L'instruction Time-In-Force IOC (Immediate-Or-Cancel) :** Elle ordonne au Bridge d'exécuter immédiatement la totalité ou une fraction de l'ordre au prix disponible, et d'annuler instantanément le solde non exécuté. Cela empêche le stockage de l'ordre dans un carnet d'ordres statique si le prix a déjà bougé.
- **Le typage de l'ordre en MARKET_RANGE :** Contrairement à un ordre `MARKET` standard qui force l'exécution quel que soit le glissement de prix (Slippage), l'ordre `MARKET_RANGE` spécifie une barrière de prix stricte. Si la phase de Last Look de 40 ms pousse le prix en dehors de la limite tolérée, la passerelle applicative cTrader annule l'ordre à la volée, protégeant ainsi l'infrastructure d'une exécution à perte.

4. Code Source Rust : Construction d'un Ordre Protégé face au Last Look

Le bloc de code suivant illustre l'implémentation industrielle en Rust pour assembler une structure d'ordre Open API conforme, intégrant le suivi d'identifiant et la protection de champ de slippage (exprimé en unités de pips de manière normalisée) :

```rust
use std::time::Instant;
use std::sync::atomic::{AtomicU64, Ordering};

// Compteur atomique global à haute performance pour générer des IDs uniques sans lock
static ORDER_COUNTER: AtomicU64 = AtomicU64::new(1);

#[derive(Debug, Clone)]
pub struct LocalTradeTracker {
    pub sent_time: Instant,
    pub symbol_id: u64,
    pub volume: u64,
}

/// Construit la structure brute de requête Protobuf pour cTrader Open API
/// Optimisé pour limiter les allocations de mémoire vive sur le chemin critique
pub fn build_protected_arbitrage_order(
    ctrader_account_id: i64,
    symbol_id: u64,
    volume_units: u64,
    trade_side: i32, // 1 = BUY, 2 = SELL
    max_slippage_pips: f64
) -> (String, Vec) {

    // 1. Génération de l'identifiant unique de corrélation
    let current_id = ORDER_COUNTER.fetch_add(1, Ordering::SeqCst);
    let client_msg_id = format!("ARB_{}_{}", Instant::now().elapsed().as_micros(), current_id);

    // 2. Initialisation et configuration de la structure binaire native (Pseudo-code structurel Protobuf)
    // Dans l'implémentation réelle, ces structures sont auto-générées via la caisse 'prost' ou 'protobuf-codegen'
    let mut order_req = ProtoOANewOrderReq::new();
    order_req.ctrader_account_id = ctrader_account_id;
    order_req.symbol_id = symbol_id;
    order_req.order_type = ProtoOAOrderType::MARKET_RANGE; // Forcer le mode de contrôle de zone de prix
    order_req.trade_side = trade_side;
    order_req.volume = volume_units;

    // Configuration critique pour court-circuiter le Last Look néfaste :
    order_req.time_in_force = ProtoOATimeInForce::IMMEDIATE_OR_CANCEL; // Instruction ECN IOC
    order_req.slippage_in_pips = Some(max_slippage_pips);             // Barrière de slippage max
    order_req.client_msg_id = Some(client_msg_id.clone());            // Clé de corrélation asynchrone

    // Encodage binaire direct de la structure Protobuf dans un tampon de vecteurs d'octets
    let mut protobuf_payload = Vec::with_capacity(128);
    order_req.encode(&mut protobuf_payload).expect("Échec critique de sérialisation Protobuf");
```

```
    (client_msg_id, protobuf_payload)
}
```

5. Matrice Opérationnelle de Gestion des Événements Asynchrones

La tâche réseau asynchrone écoute le flux de données en continu. À la fin de la période de Last Look, la décision finale prise par l'écosystème de liquidité doit être traitée instantanément selon la matrice de contingence stricte ci-dessous :

Événement Reçu (Status)	Cause Structurelle	Impact Temporel	Action Immédiate du Code Rust
`FILLED`	Ordre accepté et exécuté par le LP dans la limite de prix.	Reçu à T + ~42 ms. Il reste ~107 ms de fenêtre.	Armer immédiatement le décompte de sortie. Déclencher la surveillance active du prix pour la fermeture.
`REJECTED`	Le LP a utilisé son Last Look pour refuser le trade ou flux jugé toxique.	Reçu à T + ~42 ms (ou avant si le Bridge rejette à T+10ms).	Nettoyer la table mémoire (`pending_orders.remove()`). Si arbitrage croisé : envoyer ordre de couverture d'urgence.
`EXPIRED`	L'instruction IOC a expiré car le prix a dépassé le `MARKET_RANGE`.	Reçu à T + ~42 ms. Aucun engagement de capital.	Annuler immédiatement le cycle de l'algorithme. Pas d'action de couverture nécessaire. Sécurité préservée.

Note de conformité et avertissement de risque : Les techniques d'arbitrage de latence sur des infrastructures Open API grand public sont hautement exposées aux politiques de filtrage des courtiers. L'apparition répétée de rejets ou d'ordres de type `EXPIRED` indique que les algorithmes de détection du courtier ont classé le flux comme "Toxic Flow", ce qui peut conduire à l'application de délais artificiels de traitement (Latency Plugins) par le serveur de gestion des risques.